

应用数学导论大作业汇报

崔正平

北京大学数学科学学院

2026.06

采用有限元离散格式求解区域 $\Omega = (0, 1) \times (0, 1)$ 上的变系数 Poisson 方程:

$$\begin{cases} -\nabla \cdot (\kappa \nabla u) = f, & (x, y) \in \Omega, \\ u = g, & (x, y) \in \partial\Omega. \end{cases}$$

- 实现 MIONet 神经算子, 取训练数据网格尺寸 $N_{\text{train}} = 32$.
- 实现基于 MIONet 的混合迭代, 取测试数据网格尺寸 $N_{\text{test}} = 32, 64$, 在未参与训练的测试样本中选取一组测试函数 (κ, f, g) , 分别采用各种传统迭代和混合迭代求解问题

- 神经网络训练硬件配置 (AutoDL 租赁)
 - GPU: RTX 3090 (24GB)
 - CPU: 14 vCPU Intel(R) Xeon(R) Gold 6330 CPU @ 2.00GHz
 - 内存: 90GB
- 混合迭代测试硬件配置 (本地)
 - CPU: Intel(R) Core(TM) Ultra 5 125H
 - 内存: 32GB

系统和环境配置

- 系统: Linux (本地 Windows 11 系统通过 WSL2 运行虚拟机)
- 环境:
 - Python 3.12 (ubuntu 22.04)
 - PyTorch 2.8.0
 - CUDA 12.8
- 依赖库 (均可采用最新版本):

● torch	● sys
● scipy	● random
● numpy	● time
● matplotlib	● datetime
● yaml	● itertools
● os	● warnings
● sklearn	

网络架构和参数、激活函数和损失函数

- 网络架构:
 - κ -分支网络: 全连接网络 (FNN), 4 层中间层, 仅输出层无偏置, 有激活函数
 - f, g -分支网络: 单层线性层, 无偏置, 无激活函数
 - 主干网络: 全连接网络 (FNN), 4 层中间层, 有偏置, 有激活函数
- 网络参数:
 - 特征维度 p : 128
 - 中间层宽度: 256
- 激活函数: $\text{ReLU}(x) = \max(0, x)$
- 损失函数: 均方误差 (Mean Squared Error, MSE)

训练超参数设置

- 批次大小 (Batch Size): 64
- 训练总轮数 (Epoch): 300
- 优化器: AdamW
- 学习率调度策略: CosineAnnealingLR(余弦退火)
- 学习率: 0.001
- 权重衰减率: 0.0001
- 随机数种子: 0

- 系数生成, 基于高斯过程 (Gaussian Process):
 - 扩散系数 κ 与右端项 f : 采用基于 RBF 径向基核的二维高斯随机场生成, 并对 κ 进行了阈值截断处理 ($\kappa \geq 0.1$)
 - 边界条件 g : 采用基于 ExpSineSquared 周期核的一维高斯过程生成
- 参考解生成: 每个样本的数值精确解 u 均调用基于稀疏 LU 分解的高精度稀疏线性方程组求解器 `scipy.sparse.linalg.spsolve`
- 数据集规模与网格分辨率:
 - 训练集 (Train): 6400 个样本, 网格尺寸 $N_{\text{train}} = 32$
 - 验证集 (Val): 1600 个样本, 用于检验 MIONet 神经算子训练效果, 网格尺寸 $N_{\text{val}} = 32$
 - 测试集 (Test): 1 个样本, 用于混合迭代测试以及其他实验, 网格尺寸 $N_{\text{test}} \in \{32, 64\}$
- 随机数种子: 0

MIONet 神经算子训练结果

- 采用**相对 L^2 误差**衡量 MIONet 神经算子的精度
- 在不同批次大小 (Batch Size) 下的训练效果对比 (由于 GPU 的极高并行, 训练耗时基本与批次大小成反比):

批次大小	训练平均相对误差	测试平均相对误差
32	3.2224%	3.6077%
64	2.0349%	2.2294%
128	3.2778%	3.4528%

混合迭代测试设置

- 对于混合 (Hybrid) 迭代法, 每 $K = 40$ 次传统迭代法后进行 1 次 MIONet 神经算子校正
- 停机条件: 残差的 L^2 范数小于 10^{-12}
- 传统迭代采用无矩阵 (Matrix-free) 更新格式
- 测试耗时基于原生 Python 环境测定, 绝对耗时受底层 Python 的 GIL 锁及解释器开销影响显著, 也可类似原始论文采用 C++ 结合 LibTorch 部署, 或者采用 Numba JIT 编译消除数据通信开销

传统迭代法

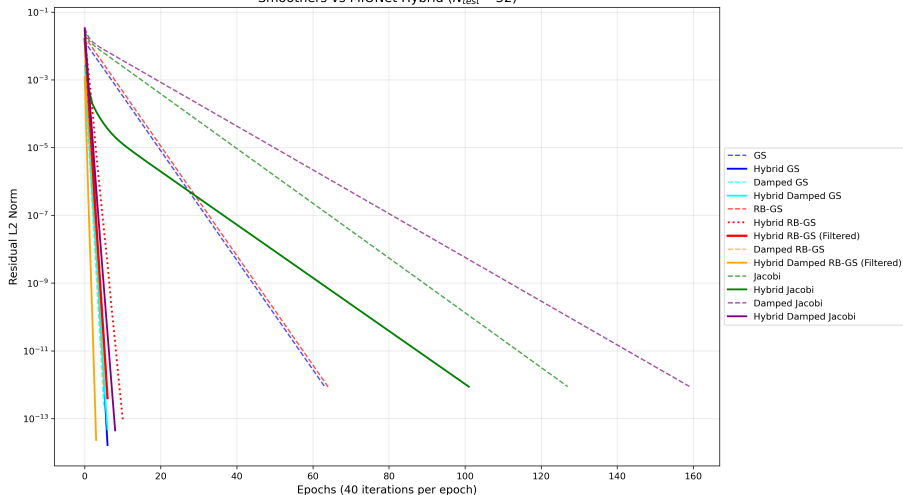
- 每种传统迭代法对于每个方格的更新逻辑, 都是根据当前与它相邻的 4 个方格内的旧值及其系数计算得到新值, 并与旧值按照松弛因子进行加权, 但方格更新的顺序互不相同:
 - GS 迭代: 按照行优先的字典序逐一更新, 存在强数据依赖, 只能严格串行
 - RB-GS 迭代: 将所有方格按照奇偶性红黑染色, 然后先同时更新所有红格, 再同时更新所有黑格, 可以并行
 - Jacobi 迭代: 同时更新所有方格, 可以并行
- 松弛因子:
 - GS 迭代的松弛因子取成 1.8
 - Jacobi 迭代的松弛因子取成 0.8
- 对于 RB-GS 迭代, 还可以引入 Filter (滤波) 操作, 在每个大循环内通过较少次数的 Damped Jacobi 迭代磨光红格与黑格之间的高频误差, 具体设置:
 - Hybrid RB-GS (Filtered): 一次大循环内 36 次 RB-GS + 4 次 Damped Jacobi
 - Hybrid Damped RB-GS (Filtered): 一次大循环内 28 次 RB-GS + 12 次 Damped Jacobi

混合迭代测试结果 ($N_{\text{test}} = 32$)

迭代法	大循环数	总耗时 (s)
GS	64	2.7537
Hybrid GS	7	0.6062
Damped GS	6	0.2537
Hybrid Damped GS	7	0.3287
RB-GS	65	0.1444
Hybrid RB-GS	11	0.0674
Hybrid RB-GS (Filtered)	7	0.0354
Damped RB-GS	6	0.0131
Hybrid Damped RB-GS (Filtered)	4	0.0197
Jacobi	128	0.0740
Hybrid Jacobi	102	0.3097
Damped Jacobi	160	0.1030
Hybrid Damped Jacobi	9	0.0317

混合迭代残差曲线 ($N_{\text{test}} = 32$)

Smoothers vs MIONet Hybrid ($N_{\text{test}} = 32$)

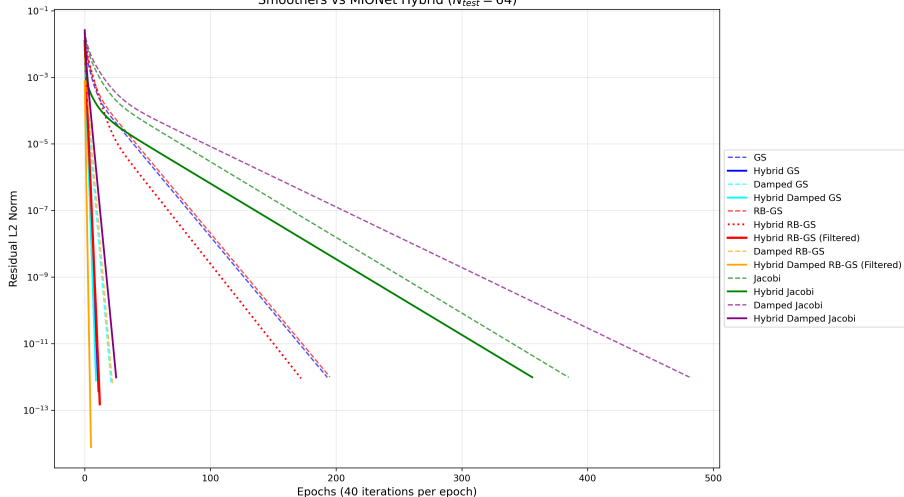


混合迭代测试结果 ($N_{\text{test}} = 64$)

迭代法	大循环数	总耗时 (s)
GS	194	33.8464
Hybrid GS	12	2.4754
Damped GS	22	3.9158
Hybrid Damped GS	10	1.9521
RB-GS	196	0.5832
Hybrid RB-GS	173	2.4346
Hybrid RB-GS (Filtered)	13	0.1731
Damped RB-GS	23	0.0702
Hybrid Damped RB-GS (Filtered)	6	0.0875
Jacobi	386	0.4937
Hybrid Jacobi	357	4.4638
Damped Jacobi	482	0.5833
Hybrid Damped Jacobi	26	0.3162

混合迭代残差曲线 ($N_{\text{test}} = 64$)

Smoothers vs MIONet Hybrid ($N_{\text{test}} = 64$)



混合迭代测试结果初步分析

- 收敛速度:

- GS 迭代的 MIONet 加速效果显著
- Damped GS 迭代在 $N_{\text{test}} = 32$ 时 MIONet 没有加速效果, 而在 $N_{\text{test}} = 64$ 时有一定的加速效果
- RB-GS 迭代在 $N_{\text{test}} = 32$ 时 MIONet 加速效果显著, 而在 $N_{\text{test}} = 64$ 时 MIONet 有一定的加速效果
- RB-GS 迭代 (Filtered) 的 MIONet 加速效果显著
- Damped RB-GS 迭代 (Filtered) 的 MIONet 有一定的加速效果
- Jacobi 迭代的 MIONet 有一定的加速效果
- Damped Jacobi 迭代的 MIONet 加速效果显著

- 耗时:

- 并行能显著加速
- 大循环数呈现线性影响
- MIONet 神经算子校正也会消耗一些时间, 在大循环数较小时影响较大

混合迭代测试结果进一步分析

- GS 迭代和 Damped Jacobi 迭代能较好地消除高频误差 (磨光性), 与具有**频谱偏置**(Spectral Bias) 特性、擅长拟合低频分量的 MIONet 形成互补
- GS 迭代和 RB-GS 迭代的收敛率**几乎相同**, 但后者可以并行加速, 效率很高, 然而 Jacobi 迭代的收敛率偏大, 尽管它也可以并行加速
- MIONet 的加速效果越好说明该传统迭代法的磨光性越好, Damped Jacobi 迭代的磨光性显著好于 Jacobi 迭代
- Filter 操作能结合 (Damped) RB-GS 迭代的较快收敛速度和可并行性, 以及 Damped Jacobi 迭代的较好的磨光性
- MIONet 作为**无穷维**神经算子具有**网格无关性** (Mesh-free): 在 $N_{\text{train}} = 32$ 训练, 可直接对 $N_{\text{test}} = 64$ 的**高分辨率网格**进行**插值预测与残差校正**, 这是因为 MIONet 的主干网络学习的是连续函数空间上的坐标映射
- 在 $N_{\text{test}} = 64$ 时, 最优的 Hybrid Damped RB-GS(Filtered) 迭代相比 GS 迭代, 在**收敛速度**上加速了 **32 倍**, 在**耗时**上加速了 **386 倍**

谢谢!